SE LAB 1 — Requirements Engineering & UML Use-Case Modelling

Internal Microservice Catalog & Health Portal

Developer Tools & IT Operations
Primary roles: DevOps Engineer and System Architect

Deliverables	Requirements Table • UML Use-Case Diagram • Use-Case Flow
Repository	https://github.com/Aryan4806/SE-LABS-PES1UG24CS086
Repository status	Public and accessible at verification time.

Prepared from the completed project repository and the Lab 1 submission structure.

1. Requirements Specification

1.1 Functional Requirements

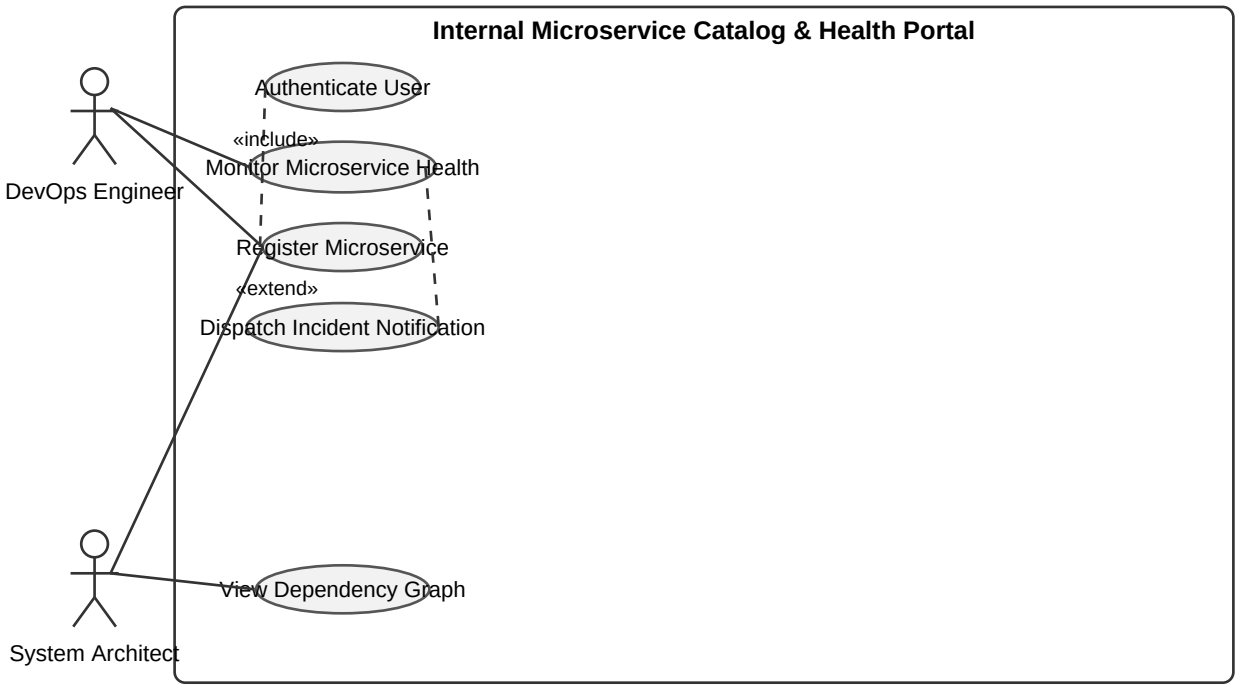
ID	Priority	Description	Acceptance Criteria	Rationale
FR-001	High	The system shall ping microservice health endpoints at 30-second intervals and compute rolling 24-hour service availability percentages.	Pass: Downtime recorded and alert dispatched on 3 consecutive failed health probes. Fail: Unreachable service marked healthy.	Essential for real-time visibility and prompt recovery during outages.
FR-002	High	The system shall maintain an interactive dependency map visualizing service-to-service communication.	Pass: Adding/removing a dependency updates graph within 5 seconds. Fail: Graph fails to render broken links.	Helps System Architects analyze cascade failure risks.
FR-003	Medium	The system shall aggregate and display OpenAPI/Swagger documentation for all registered microservices.	Pass: API specs auto-render upon service registration. Fail: Outdated or missing spec rendered without warning.	Centralizes API specs for seamless developer onboarding.
FR-004	High	The system shall send instant notifications via Webhooks/Slack to DevOps Engineers when service failure is detected.	Pass: Alert payload delivered within 2 seconds of probe failure threshold. Fail: Alerts dropped or delayed over 1 minute.	Reduces Mean Time to Detect (MTTD) operational incidents.
FR-005	Low	The system shall allow System Architects to tag and categorize microservices by environment (Dev, Staging, Prod).	Pass: Filtering catalog by tag returns correct environment subset. Fail: Untagged or incorrectly isolated services appear in query.	Ensures logical isolation and environment management.

1.2 Non-Functional Requirements

ID	Type	Description	Priority	Acceptance Criteria
NFR-001	Performance & Security	The service catalog dependency graph viewer must render interactive node architectures containing up to 200 services smoothly.	High	Pass: Latency stays below 2 seconds during peak load. Fail: Graph stutters or crashes browser canvas.
NFR-002	Security	Role-Based Access Control (RBAC) must restrict service registry modifications to authorized DevOps personnel only.	High	Pass: Non-admin write operations return HTTP 403 Forbidden. Fail: Unauthenticated user modifies service endpoints.

2. UML Use-Case Diagram

The following diagram represents the actors, primary use cases, and the «include» / «extend» relationships defined in the completed repository.



Actor / Use Case	Details
DevOps Engineer	Monitor Microservice Health; Register Microservice
System Architect	View Dependency Graph; Register Microservice
Monitor Microservice Health	Automated health probing and rolling availability tracking.
Register Microservice	Adds a service to the catalog; includes user authentication.
View Dependency Graph	Displays service-to-service communication and architecture relationships.
Authenticate User	Mandatory authentication sub-flow included by service registration.
Dispatch Incident Notification	Conditional alert behavior extending health monitoring when failures meet the threshold.

3. Use-Case Flow Specification

UC-001: Monitor Microservice Health & Alert

Use Case ID	UC-001
Use Case Name	Monitor Microservice Health & Alert
Primary Actor	DevOps Engineer

Preconditions

1. Microservice health check URL is registered in the catalog.
2. Notification channels (e.g., Slack Webhook) are configured.

Postconditions

1. Service health metrics are updated in the rolling 24-hour availability database.
2. Failure logs and alert messages are dispatched if failure thresholds are met.

Main Success Scenario

Step	Actor	Action
1	System	Automated pinger sends HTTP GET health request to target microservice at 30-second intervals.
2	Target Microservice	Returns 200 OK HTTP status code within the expected timeout window.
3	System	Logs the successful health-check metric.
4	System	Updates the rolling 24-hour availability percentage displayed on the dashboard.

Alternate Flow A — Microservice Health Check Fails

A1. At Step 2, the target microservice fails to respond or returns a non-200 HTTP code.
A2. System flags the probe attempt as failed and increments the failure counter.
A3. System re-probes the service immediately.
A4. If 3 consecutive probes fail, the system updates the microservice status to CRITICAL/DOWN.
A5. System triggers the automated alert rule, corresponding to «extend» UC-002: Dispatch Incident Notification.
A6. System logs the downtime event timestamp and resumes the background pinging routine.

Repository Verification

The repository is public and was accessible when checked. It contains the requirements, rendered use-case diagram, PlantUML source, and use-case specification files. https://github.com/Aryan4806/SE-LABS-PES1UG24CS086

Source alignment note: The content above is based directly on the completed repository files and the supplied Lab 1 submission structure.